
Course Instructor: **Md. Asif Khan Rifat**
Course: **CSE103 (Structured Programming)**

Lab: **07**

Topics Today

1. Strings

Strings in C are arrays of characters ending with a null terminator ('\0'). The <string.h> library provides several built-in functions to simplify common string operations.

Key String Functions in <string.h>

1. **strlen(s)**
Returns the length of the string s (excluding the null terminator).
Example: `strlen("hello") → 5`
2. **strlwr(s) / strupr(s)**
Converts all characters in the string s to lowercase/uppercase.
Example: `strlwr("Hello") → "hello"`
3. **strrev(s)**
Reverses the string s.
Example: `strrev("hello") → "olleh"`
4. **strcpy(dest, src)**
Copies the content of src into dest.
Example:

```
char src[] = "hello", dest[10];  
strcpy(dest, src); // dest now contains "hello"
```

5. **strncpy(dest, src, n)**
Copies up to n characters from src into dest.
6. **strcat(dest, src)**
Concatenates (appends) src to the end of dest.
Example:

```
char str1[20] = "hello ", str2[] = "world";  
strcat(str1, str2); // str1 now contains "hello world"
```

7. **strcmp(s1, s2)**
Compares two strings lexicographically.
 - Returns 0 if they are equal.
 - Returns a positive/negative value if $s1 > s2$ or $s1 < s2$.

Problem 1: Palindrome Checker

Write a program that takes a string as input and checks if it is a palindrome. A palindrome reads the same backward as forward. Ignore case and spaces.

Example:

Input: "A man a plan a canal Panama"

Output: "Palindrome"

Hint:

- Use `strlwr()` to convert the string to lowercase.
- Remove spaces by copying only non-space characters into a new string. Pseudocode is given below:
 1. `char str[100], result[100];`
 2. `int i, j = 0;`
 3. `printf("Enter a string: ");`
 4. `gets(str);`
 5. `for (i = 0; i < strlen(str); i++) {`
 6. `if (str[i] != ' ') {`
 7. `result[j] = str[i];`
 8. `j++;`
 9. `}`
 10. `}`
 11. `result[j] = '\0';`
 12. `//now result contains string without spaces`
- Reverse the string using `strrev()` and compare it with the original.

Problem 2: Anagram Detector

Write a program to check if two strings are anagrams of each other. Ignore case and spaces.

Example:

Input: "Listen" and "Silent"

Output: "Anagrams"

Hint:

- Use `strlwr()` to convert both strings to lowercase.
- Remove spaces from both strings. See Problem 1 hint to remove spaces.
- Sort both strings alphabetically. Sorting of a string pseudocode is as follows:
 1. `for(i=0; i<strlen(str)-1; i++){`
 2. `for(j=i+1; j<strlen(str); j++){`
 3. `if(str[i]>str[j]){`
 4. `temp = str[i];`
 5. `str[i] = str[j];`
 6. `str[j] = temp;`
 7. `}`
 8. `}`
 9. `}`
- Compare each character of both string at same index after sorting.

Problem 3: Word, letter, vowel, consonant Counter

Write a program to count the total words, letters, vowels and consonants in a given string. Ignore case and punctuation marks.

Example:

Input: "Hello world! Hello everyone."

Output:

Word: 4

Vowel: 9

Consonant: 14

Letter: 23

Hint:

- Use `strlwr()` to convert the string to lowercase.
- Remove punctuation/non-alphabetic characters using a loop. Pseudocode is below:

```
1.     for (i = 0; str[i] != '\0'; i++) {
2.         // Check if the character is not a non-alphabetic letter
3.         if ((str[i] >= 'A' && str[i] <= 'Z') || (str[i] >= 'a' &&
str[i] <= 'z') || (str[i] == ' ') || (str[i] >= '0' && str[i] <=
'9')) {
4.             result[j] = str[i];
5.             j++;
6.         }
7.     }
```

- Count the words, letters, vowels and consonants. See lecture slide.

Problem 4: Substring Search

Write a program that searches for a substring in a string and prints its starting index.

Example:

Input:

- String: "the quick brown fox jumps over the lazy dog"
 - Target: "brown"
- Output: "brown starts at index 10"

Hint:

- See lecture slide